End-Sem Lab Exam — Complete Preparation

1. Aim → 2. Software Requirements → 3. Structure → 4. Program → 5. Postman Data Posting → 6. Output

---

EXPERIMENT 1 — CREATE AND INSERT PRODUCT

1. Aim

To create a Product entity and implement a REST API to insert product details into the database using Spring Boot and test the API using Postman.

2. Software Requirements

- Java JDK
- Spring Boot
- Spring Data JPA
- MySQL
- Maven
- Postman
- IntelliJ IDEA / Eclipse

3. Structure

```
src/main/java
 └── com.example.ecommerce
      ├── controller
      │    └── ProductController.java
      ├── entity
      │    └── Product.java
      ├── repository
      │    └── ProductRepository.java
      └── EcommerceApplication.java
```

4. Program

Product.java

```
package com.example.ecommerce.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;

@Entity
public class Product {

    @Id
    private Long productId;

    private String productName;
    private String category;
    private String description;
    private double price;
    private int stock;
    private String imageUrl;
    private double rating;
    private String status;

    public Long getProductId() {
        return productId;
    }

    public void setProductId(Long productId) {
        this.productId = productId;
    }

    public String getProductName() {
        return productName;
    }

    public void setProductName(String productName) {
        this.productName = productName;
    }

    public String getCategory() {
        return category;
    }

    public void setCategory(String category) {
        this.category = category;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
```

```
        this.price = price;
    }

    public int getStock() {
        return stock;
    }

    public void setStock(int stock) {
        this.stock = stock;
    }

    public String getImageUrl() {
        return imageUrl;
    }

    public void setImageUrl(String imageUrl) {
        this.imageUrl = imageUrl;
    }

    public double getRating() {
        return rating;
    }

    public void setRating(double rating) {
        this.rating = rating;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }
}
```

ProductRepository.java

```
package com.example.ecommerce.repository;

import com.example.ecommerce.entity.Product;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ProductRepository
        extends JpaRepository<Product, Long> {
}
```

ProductController.java

```
package com.example.ecommerce.controller;

import com.example.ecommerce.entity.Product;
import com.example.ecommerce.repository.ProductRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
```

```
@RestController
@RequestMapping("/products")
public class ProductController {

    @Autowired
    private ProductRepository repository;

    @PostMapping
    public Product addProduct(@RequestBody Product product) {
        return repository.save(product);
    }
}
```

EcommerceApplication.java

```
package com.example.ecommerce;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class EcommerceApplication {

    public static void main(String[] args) {
        SpringApplication.run(EcommerceApplication.class, args);
    }
}
```

application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/ecommerce
spring.datasource.username=root
spring.datasource.password=root

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect

server.port=8080
```

5. Postman Data Posting

Method

```
POST
```

URL

```
http://localhost:8080/products
```

Body → raw → JSON

```
{
  "productId": 101,
  "productName": "Laptop",
  "category": "Electronics",
  "description": "HP Laptop",
  "price": 55000,
  "stock": 10,
  "imageUrl": "laptop.jpg",
  "rating": 4.5,
  "status": "Available"
}
```

For the second product:

```
{
  "productId": 102,
  "productName": "Mobile",
  "category": "Electronics",
  "description": "Samsung Mobile",
  "price": 25000,
  "stock": 20,
  "imageUrl": "mobile.jpg",
  "rating": 4.3,
  "status": "Available"
}
```

6. Output

```
{
  "productId": 101,
  "productName": "Laptop",
  "category": "Electronics",
  "description": "HP Laptop",
  "price": 55000,
  "stock": 10,
  "imageUrl": "laptop.jpg",
  "rating": 4.5,
  "status": "Available"
}
```

EXPERIMENT 2 — CREATE AND RETRIEVE PRODUCT DETAILS

1. Aim

To implement REST APIs to retrieve a single product using productId and retrieve all products from the database.

2. Software Requirements

- Java JDK
- Spring Boot
- Spring Data JPA
- MySQL
- Maven
- Postman

3. Structure

```
src/main/java
 └── com.example.ecommerce
      ├── controller
      │    └── ProductController.java
      ├── entity
      │    └── Product.java
      ├── repository
      │    └── ProductRepository.java
      └── EcommerceApplication.java
```

4. Program

ProductController.java

```
package com.example.ecommerce.controller;

import com.example.ecommerce.entity.Product;
import com.example.ecommerce.repository.ProductRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/products")
public class ProductController {

    @Autowired
    private ProductRepository repository;

    @GetMapping("/{id}")
    public ResponseEntity<Product> getProduct(
            @PathVariable Long id) {

        return repository.findById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }
```

```
    @GetMapping
    public List<Product> getAllProducts() {
        return repository.findAll();
    }
}
```

`Product.java`, `ProductRepository.java` and `EcommerceApplication.java` remain the same as Experiment 1.

5. Postman Data Posting

Get Single Product

```
GET
http://localhost:8080/products/101
```

Get All Products

```
GET
http://localhost:8080/products
```

6. Output

Single Product

```
{
  "productId": 101,
  "productName": "Laptop",
  "category": "Electronics",
  "description": "HP Laptop",
  "price": 55000,
  "stock": 10,
  "imageUrl": "laptop.jpg",
  "rating": 4.5,
  "status": "Available"
}
```

All Products

```
[
  {
    "productId": 101,
    "productName": "Laptop",
    "price": 55000,
    "stock": 10
  },
  {
    "productId": 102,
    "productName": "Mobile",
    "price": 25000,
    "stock": 20
```

```
  }
]
```

EXPERIMENT 3 — CREATE AND UPDATE PRODUCT DETAILS

1. Aim

To implement a REST API to update the details of a specific product using its productId.

2. Software Requirements

- Java JDK
- Spring Boot
- Spring Data JPA
- MySQL
- Maven
- Postman

3. Structure

```
src/main/java
 └── com.example.ecommerce
      ├── controller
      │    └── ProductController.java
      ├── entity
      │    └── Product.java
      ├── repository
      │    └── ProductRepository.java
      └── EcommerceApplication.java
```

4. Program

ProductController.java

```
package com.example.ecommerce.controller;

import com.example.ecommerce.entity.Product;
import com.example.ecommerce.repository.ProductRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/products")
public class ProductController {
```

```
    @Autowired
    private ProductRepository repository;

    @PutMapping("/{id}")
    public Product updateProduct(
            @PathVariable Long id,
            @RequestBody Product product) {

        Product existing =
                repository.findById(id).orElse(null);

        if (existing != null) {

            existing.setPrice(product.getPrice());
            existing.setStock(product.getStock());
            existing.setStatus(product.getStatus());

            return repository.save(existing);
        }

        return null;
    }
}
```

5. Postman Data Posting

Method

```
PUT
```

URL

```
http://localhost:8080/products/101
```

Body

```
{
  "price": 50000,
  "stock": 25,
  "status": "Available"
}
```

6. Output

```
{
  "productId": 101,
  "productName": "Laptop",
  "category": "Electronics",
  "description": "HP Laptop",
  "price": 50000,
  "stock": 25,
```

```
  "imageUrl": "laptop.jpg",
  "rating": 4.5,
  "status": "Available"
}
```

EXPERIMENT 4 — CREATE AND DELETE SPECIFIC PRODUCT

1. Aim

To implement an API to delete a specific product from the database using its productId.

2. Software Requirements

- Java JDK
- Spring Boot
- Spring Data JPA
- MySQL
- Maven
- Postman

3. Structure

```
src/main/java
 └── com.example.ecommerce
      ├── controller
      │    └── ProductController.java
      ├── entity
      │    └── Product.java
      ├── repository
      │    └── ProductRepository.java
      └── EcommerceApplication.java
```

4. Program

ProductController.java

```
package com.example.ecommerce.controller;

import com.example.ecommerce.entity.Product;
import com.example.ecommerce.repository.ProductRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
```

```
@RequestMapping("/products")
public class ProductController {

    @Autowired
    private ProductRepository repository;

    @DeleteMapping("/{id}")
    public String deleteProduct(
            @PathVariable Long id) {

        if (repository.existsById(id)) {

            repository.deleteById(id);

            return "Product deleted successfully";
        }

        return "Product not found";
    }
}
```

5. Postman Data Posting

Method

```
DELETE
```

URL

```
http://localhost:8080/products/101
```

No body is required.

6. Output

```
Product deleted successfully
```

If the product does not exist:

```
Product not found
```

EXPERIMENT 5 — USER SIGNUP AND SIGNIN

1. Aim

To create a User entity and implement signup and signin functionality for the E-Commerce application.

2. Software Requirements

- Java JDK
- Spring Boot
- Spring Data JPA
- MySQL
- Maven
- Postman

3. Structure

```
src/main/java
 └── com.example.ecommerce
      ├── controller
      │    └── UserController.java
      ├── entity
      │    └── User.java
      ├── repository
      │    └── UserRepository.java
      └── EcommerceApplication.java
```

4. Program

User.java

```
package com.example.ecommerce.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;

@Entity
public class User {

    @Id
    private Long userId;

    private String name;
    private String email;
    private String password;
    private String role;

    public Long getUserId() {
        return userId;
    }

    public void setUserId(Long userId) {
        this.userId = userId;
```

```
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getRole() {
        return role;
    }

    public void setRole(String role) {
        this.role = role;
    }
}
```

UserRepository.java

```
package com.example.ecommerce.repository;

import com.example.ecommerce.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository
        extends JpaRepository<User, Long> {

    User findByEmail(String email);
}
```

UserController.java

```
package com.example.ecommerce.controller;

import com.example.ecommerce.entity.User;
import com.example.ecommerce.repository.UserRepository;
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/users")
public class UserController {

    @Autowired
    private UserRepository repository;

    @PostMapping("/signup")
    public User signup(@RequestBody User user) {

        return repository.save(user);
    }

    @PostMapping("/signin")
    public String signin(@RequestBody User user) {

        User existing =
                repository.findByEmail(user.getEmail());

        if (existing != null &&
            existing.getPassword()
                    .equals(user.getPassword())) {

            return "Signin successful";
        }

        return "Invalid credentials";
    }
}
```

5. Postman Data Posting

Signup

```
POST
http://localhost:8080/users/signup
{
  "userId": 1,
  "name": "Joel",
  "email": "joel@gmail.com",
  "password": "12345",
  "role": "USER"
}
```

Signin

```
POST
http://localhost:8080/users/signin
{
  "email": "joel@gmail.com",
  "password": "12345"
}
```

6. Output

Signup

```
{
  "userId": 1,
  "name": "Joel",
  "email": "joel@gmail.com",
  "password": "12345",
  "role": "USER"
}
```

Signin

```
Signin successful
```

EXPERIMENT 6 — JWT GENERATION AFTER SIGNIN

1. Aim

To implement JWT-based authentication and generate a JWT token after successful user signin.

The PDF specifically requires registered email/password validation, JWT generation, returning the token, and testing it through Postman. EndSem Lab Model QP Experiments

2. Software Requirements

- Java JDK
- Spring Boot
- Spring Data JPA
- Spring Security
- JWT
- MySQL
- Maven
- Postman

3. Structure

```
src/main/java
 └── com.example.ecommerce
      ├── controller
      │    └── UserController.java
```

```
├── entity
│   └── User.java
├── repository
│   └── UserRepository.java
├── service
│   └── JwtService.java
└── EcommerceApplication.java
```

4. Program

JwtService.java

```
package com.example.ecommerce.service;

import org.springframework.stereotype.Service;

import java.util.Base64;

@Service
public class JwtService {

    public String generateToken(String email) {

        String header =
                Base64.getEncoder()
                        .encodeToString(
                                "{\"alg\":\"none\",\"typ\":\"JWT\"}"
                                .getBytes());

        String payload =
                Base64.getEncoder()
                        .encodeToString(
                                ("{\"email\":\"" + email + "\"}")
                                .getBytes());

        return header + "." + payload + ".signature";
    }
}
```

UserController.java

```
package com.example.ecommerce.controller;

import com.example.ecommerce.entity.User;
import com.example.ecommerce.repository.UserRepository;
import com.example.ecommerce.service.JwtService;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/users")
public class UserController {
```

```
    @Autowired
    private UserRepository repository;

    @Autowired
    private JwtService jwtService;

    @PostMapping("/signin")
    public String signin(@RequestBody User user) {

        User existing =
                repository.findByEmail(user.getEmail());

        if (existing != null &&
            existing.getPassword()
                    .equals(user.getPassword())) {

            return jwtService.generateToken(
                    existing.getEmail());
        }

        return "Invalid credentials";
    }
}
```

Exam note: This demonstrates the JWT token-generation flow. In a real application, use a properly signed JWT library and secret key.

5. Postman Data Posting

Method

```
POST
```

URL

```
http://localhost:8080/users/signin
```

Body

```
{
  "email": "joel@gmail.com",
  "password": "12345"
}
```

6. Output

```
eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0
.
eyJlbWFpbCI6ImpvZWxAZ21haWwuY29tIn0
.
signature
```

The signin API successfully returns a JWT token after valid credentials.

EXPERIMENT 7 — EMAIL NOTIFICATION

1. Aim

To implement an email notification feature that sends a confirmation email to the registered user after successful signup.

The PDF requires `JavaMailSender`, SMTP configuration, user email acceptance, successful account creation, email sending, and verification. EndSem Lab Model QP Experiments

2. Software Requirements

- Java JDK
- Spring Boot
- Spring Data JPA
- JavaMailSender
- SMTP
- MySQL
- Maven
- Postman

3. Structure

```
src/main/java
 └── com.example.ecommerce
      ├── controller
      │    └── UserController.java
      ├── entity
      │    └── User.java
      ├── repository
      │    └── UserRepository.java
      ├── service
      │    └── EmailService.java
      └── EcommerceApplication.java
```

4. Program

EmailService.java

```
package com.example.ecommerce.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.mail.SimpleMailMessage;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.stereotype.Service;

@Service
public class EmailService {

    @Autowired
    private JavaMailSender mailSender;

    public void sendEmail(String to) {

        SimpleMailMessage message =
                new SimpleMailMessage();

        message.setTo(to);
        message.setSubject(
                "Registration Successful");

        message.setText(
                "Your account has been successfully created.");

        mailSender.send(message);
    }
}
```

UserController.java

```
package com.example.ecommerce.controller;

import com.example.ecommerce.entity.User;
import com.example.ecommerce.repository.UserRepository;
import com.example.ecommerce.service.EmailService;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/users")
public class UserController {

    @Autowired
    private UserRepository repository;

    @Autowired
    private EmailService emailService;

    @PostMapping("/signup")
    public User signup(@RequestBody User user) {

        User saved =
                repository.save(user);
```

```
        emailService.sendEmail(
                user.getEmail());

        return saved;
    }
}
```

application.properties

```
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=yourmail@gmail.com
spring.mail.password=your-app-password

spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
```

5. Postman Data Posting

Method

```
POST
```

URL

```
http://localhost:8080/users/signup
```

Body

```
{
  "userId": 2,
  "name": "Joel",
  "email": "joel@gmail.com",
  "password": "12345",
  "role": "USER"
}
```

6. Output

Postman

```
{
  "userId": 2,
  "name": "Joel",
  "email": "joel@gmail.com",
  "password": "12345",
  "role": "USER"
}
```

Email

```
Subject: Registration Successful

Your account has been successfully created.
```

⭐ TOMORROW'S EXAM — WHAT TO REMEMBER

Common Project Structure

For **Experiments 1–4**, remember:

```
src/main/java
 └── com.example.ecommerce
      ├── controller
      │    └── ProductController.java
      ├── entity
      │    └── Product.java
      ├── repository
      │    └── ProductRepository.java
      └── EcommerceApplication.java
```

For **Experiments 5–7**:

```
src/main/java
 └── com.example.ecommerce
      ├── controller
      │    └── UserController.java
      ├── entity
      │    └── User.java
      ├── repository
      │    └── UserRepository.java
      ├── service
      │    ├── JwtService.java
      │    └── EmailService.java
      └── EcommerceApplication.java
```

Most Important HTTP Methods

Experiment	Method	URL
Insert	`POST`	`/products`
Retrieve one	`GET`	`/products/101`
Retrieve all	`GET`	`/products`
Update	`PUT`	`/products/101`
Delete	`DELETE`	`/products/101`
Signup	`POST`	`/users/signup`

Experiment	Method	URL
Signin	`POST`	`/users/signin`
JWT Signin	`POST`	`/users/signin`

Product Fields

```
productId
productName
category
description
price
stock
imageUrl
rating
status
```

User Fields

```
userId
name
email
password
role
```

These are the **7 experiments specified in your lab PDF**, arranged in the exact order you requested. EndSem Lab Model QP Experiments